

Chapter 6: Pickup-and-Delivery Problems for Goods Transportation

Vehicle Routing: Problems, Methods, and Applications
Second Edition, 2014

Maria Battarra Jean-François Cordéau Manuel Iori

Chapter 6, pp. 161–202

Chapter Outline

- 1 Introduction
- 2 Many-to-Many Problems
- 3 One-to-Many-to-One Problems
- 4 One-to-One Problems
- 5 Problems with Loading Constraints
- 6 Conclusions and Future Research Directions
- 7 Bibliography

6.1 Introduction: What Are Pickup-and-Delivery Problems? I

Pickup-and-Delivery Problems (PDPs) constitute an important and broad family of routing problems in which goods or passengers must be transported between origins and destinations. Unlike the classical Vehicle Routing Problem (VRP) – in which goods flow only *from* a depot *to* customers – a PDP involves two-sided requests: for each request there is a **pickup location** where the item is collected and a **delivery location** where the item is dropped off.

Formal Definition of a PDP Request

A **request** r specifies:

- a **pickup node** p_r : the origin where the item must be collected,
- a **delivery node** d_r : the destination where the item must be delivered,
- a **load** q_r : the quantity (weight, volume) associated with the request,
- (optionally) **time windows** $[e_{p_r}, l_{p_r}]$ and $[e_{d_r}, l_{d_r}]$ specifying the earliest and latest service times at both endpoints.

The fundamental constraint is that the pickup must occur *before* the delivery on the same vehicle's route.

6.1 Introduction: What Are Pickup-and-Delivery Problems? II

Three structural categories. PDPs are classified according to the relationship between origins and destinations of requests:

- 1 **Many-to-Many (M-M):** Any node can be both an origin and a destination. This arises in transportation networks such as freight exchange platforms, where goods move between arbitrary pairs of nodes. Dial-a-ride problems without fixed depot constraints are special cases.
- 2 **One-to-Many-to-One (1-M-1):** Goods originate from a *single depot*, are delivered to customers, and empty or loaded vehicles return to the same depot. This models distribution networks where a vehicle may also collect goods (e.g., returned merchandise) during the same tour.
- 3 **One-to-One (1-1):** Each request specifies a *unique* pickup location and a *unique* delivery location. Items cannot be transhipped: the same vehicle that picks up an item must deliver it. This is the most constrained and the most widely studied variant for goods transportation.

6.1 Introduction: What Are Pickup-and-Delivery Problems? III

Three Categories of Pickup-and-Delivery Problems

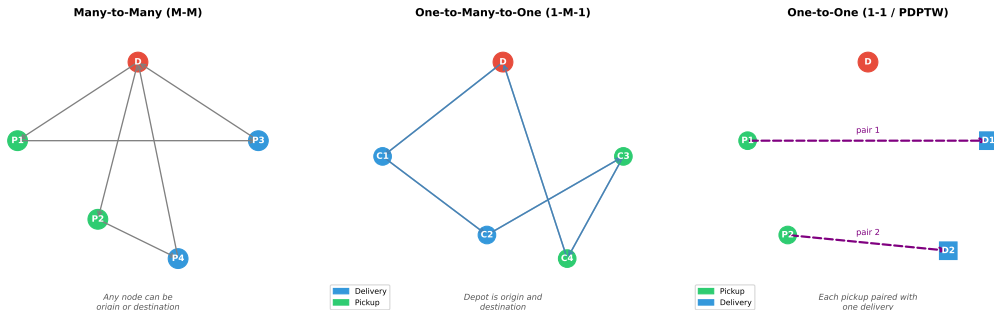


Figure: The three structural categories of PDPs. Green = pickup nodes, blue = delivery nodes, red = depot. In M-M any node is both; in 1-M-1 the depot is origin/destination; in 1-1 each pickup is paired with exactly one delivery.

6.1 Introduction: What Are Pickup-and-Delivery Problems? IV

Additional problem dimensions. Beyond the structural category, PDPs are further characterised by:

- **Fleet composition:** homogeneous (all vehicles identical) or heterogeneous (vehicles differ in capacity, cost, or type).
- **Time windows:** hard (service outside the window is forbidden) or soft (violations are penalised in the objective).
- **Loading constraints:** LIFO (Last-In First-Out), FIFO (First-In First-Out), or full three-dimensional packing feasibility.
- **Objective:** minimise total travel distance or time, minimise number of vehicles, or a weighted combination.

6.1 Introduction: What Are Pickup-and-Delivery Problems? V

Key Insight: Why PDPs Are Harder Than VRPs

In a standard VRP, feasibility checking for a route requires only that the total demand does not exceed capacity. In a PDP, feasibility also requires: (1) the pickup precedes its paired delivery (*precedence* constraint), (2) the vehicle load never exceeds capacity at any point on the route (*capacity* constraint along the tour, not just in total), and (3) all time windows are satisfied. The combination of these constraints makes PDPs significantly harder combinatorially.

Dynamic and stochastic variants. Many real-world applications involve *dynamic* PDPs in which requests arrive in real time (courier services, taxi dispatching), or *stochastic* PDPs in which demand or travel times are uncertain. This chapter focuses on the static, deterministic setting.

6.2 Many-to-Many (M-M) Problems I

In the Many-to-Many PDP, multiple origins and multiple destinations exist, and any node can serve as both an origin and a destination. The classic example is a transportation network where goods are to be moved between arbitrary pairs of nodes, typically using a fleet of vehicles.

M-M PDP: Mathematical Formulation

Let $G = (V, A)$ be a directed graph where $V = \{0, 1, \dots, n\}$ is the node set and A is the arc set. Each node $i \in V$ has a **supply** s_i (positive if it produces goods, negative if it consumes goods). We seek routes for m vehicles of capacity Q such that:

$$\min \sum_{k=1}^m \sum_{(i,j) \in A} c_{ij} x_{ij}^k$$

subject to flow conservation at every node, vehicle capacity constraints, and the requirement that all supply-demand imbalances are resolved. Here c_{ij} is the travel cost of arc (i, j) and $x_{ij}^k \in \{0, 1\}$ indicates whether vehicle k traverses arc (i, j) .

6.2 Many-to-Many (M-M) Problems II

Dial-a-Ride relaxation. A well-known special case of M-M problems is the **Dial-a-Ride Problem (DARP)**, in which users specify an origin and destination and a vehicle must transport them. In the goods context, the “passengers” are replaced by freight units. When the depot is not a fixed constraint – meaning vehicles may start and end at any node – the problem becomes a pure M-M problem.

6.2 Many-to-Many (M-M) Problems III

Single-vehicle case. Hernandez-Perez and Salazar-Gonzalez (2004) introduced the **1-PDTSP** (one-vehicle Pickup-and-Delivery Travelling Salesman Problem) for M-M settings. A complete graph $G = (V, A)$ is given; each node i has a supply s_i (positive = source, negative = sink), and a single vehicle of capacity Q must visit all nodes. The 1-PDTSP can be formulated as:

6.2 Many-to-Many (M-M) Problems IV

1-PDTSP Formulation

Decision variables:

- $x_{ij} \in \{0, 1\}$: equals 1 if arc (i, j) is traversed by the vehicle.
- $y_i \geq 0$: vehicle load upon departure from node i .

Objective:

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

Constraints:

$$\sum_j x_{ij} = 1 \quad \forall i \in V \quad (1: \text{ each node left once})$$

$$\sum_i x_{ij} = 1 \quad \forall j \in V \quad (2: \text{ each node entered once})$$

$$y_j \geq y_i + s_j \cdot x_{ij} - Q(1 - x_{ij}) \quad \forall i \neq j \quad (3: \text{ load tracking})$$

$$0 \leq y_i \leq Q \quad \forall i \in V \quad (4: \text{ capacity})$$

$$x_{ij} \in \{0, 1\} \quad (5: \text{ binary})$$

6.3 One-to-Many-to-One (1-M-1) Problems I

In the One-to-Many-to-One PDP, all goods originate from (or return to) a single depot. Vehicles depart from the depot carrying goods for delivery, visit customers to make deliveries, and may also collect goods (pickups) from customers during the same tour before returning to the depot. This structure models real-world applications such as beverage distribution (delivering full crates and collecting empty ones), or the simultaneous distribution and collection of products in retail networks.

6.3 One-to-Many-to-One (1-M-1) Problems II

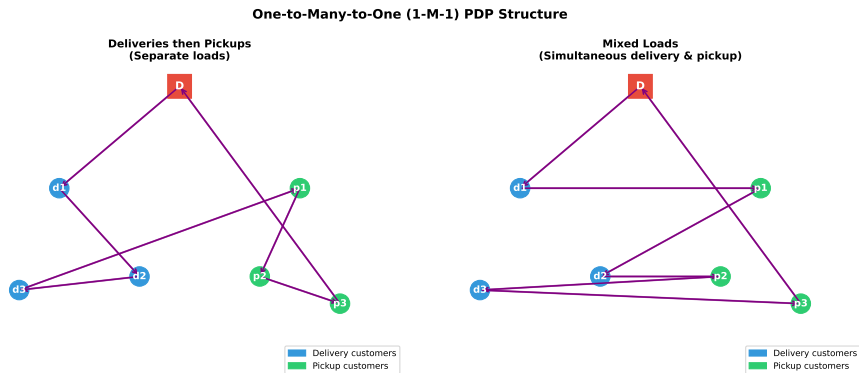


Figure: Two variants of the 1-M-1 structure. Left: deliveries precede pickups (vehicles first unload all goods, then collect). Right: mixed loads (delivery and pickup customers are interleaved on the route).

6.3 One-to-Many-to-One (1-M-1) Problems III

Two sub-cases of 1-M-1. The 1-M-1 structure can be further divided:

- ① **Deliveries then pickups:** the vehicle visits all delivery customers before visiting any pickup customer. This is a natural operational constraint when delivery and pickup goods cannot be mixed (e.g., fragile items).
- ② **Mixed loads:** delivery and pickup customers are interleaved on the route. The vehicle load at any point is the original delivery load minus goods delivered plus goods collected up to that point. The capacity constraint must hold throughout the route.

6.3 One-to-Many-to-One (1-M-1) Problems IV

The Vehicle Routing Problem with Simultaneous Pickup and Delivery (VRPSPD)

The VRPSPD is the most general form of 1-M-1. A fleet of m homogeneous vehicles, each of capacity Q , must serve n customers. Each customer i has a **delivery demand** $d_i > 0$ and a **pickup demand** $p_i \geq 0$. The vehicle must carry both when visiting customer i .

Decision variables:

- $x_{ij}^k \in \{0, 1\}$: 1 if vehicle k travels from i to j .
- $y_{ij}^k \geq 0$: delivery load on vehicle k when traversing arc (i, j) .
- $z_{ij}^k \geq 0$: pickup load on vehicle k when traversing arc (i, j) .

Objective – minimise total travel cost:

$$\min \sum_{k=1}^m \sum_{(i,j) \in A} c_{ij} x_{ij}^k$$

Core constraints: flow conservation, each customer visited exactly once, and at every arc the combined delivery and pickup loads must satisfy:

6.4.1 The PDPTW: Mathematical Formulation I

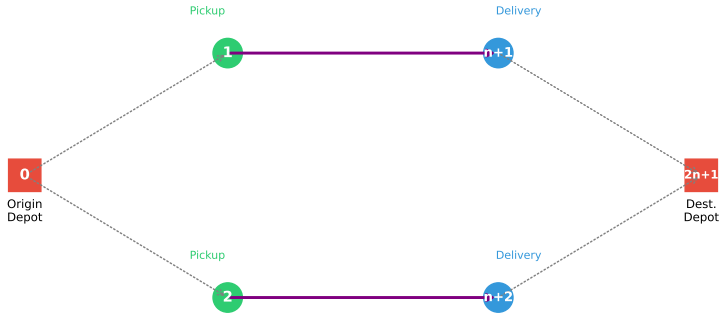
The **Pickup-and-Delivery Problem with Time Windows (PDPTW)** is the most extensively studied variant of the 1-1 PDP. In this problem, each request specifies a unique pickup location and a unique delivery location with associated time windows, and the same vehicle that picks up the goods must deliver them without intermediate transshipment.

Problem data.

- n : number of requests.
- Node set: $N = \{0, 1, \dots, 2n, 2n + 1\}$ where node 0 is the origin depot, node $2n + 1$ is the destination depot (a copy of node 0), nodes $1, \dots, n$ are pickup nodes, and nodes $n + 1, \dots, 2n$ are delivery nodes.
- Each request $i \in \{1, \dots, n\}$ consists of pickup node i and delivery node $n + i$.
- c_{ij} : travel cost from node i to node j .
- t_{ij} : travel time from node i to node j .
- $[e_i, l_i]$: time window at node i (service must begin in this interval).
- q_i : load change at node i ($q_i > 0$ for pickup nodes, $q_i < 0$ for delivery nodes).
- Q : vehicle capacity (maximum load at any point in the route).

6.4.1 The PDPTW: Mathematical Formulation II

PDP Graph: Single-Vehicle Request Graph
(Pickup nodes 1- n , Delivery nodes $n+1$ to $2n$, Depot 0 and $2n+1$)



Purple arrows = precedence: pickup must happen before its paired delivery

6.4.1 The PDPTW: Mathematical Formulation III

Figure: The request graph for the PDPTW. Pickup nodes (green) are numbered $1 \dots n$; delivery nodes (blue) are $n + 1 \dots 2n$; origin and destination depots are shown in red. Purple arrows indicate precedence: pickup i must precede delivery $n + i$ on the same route.

6.4.1 The PDPTW: Mathematical Formulation IV

PDPTW: Complete Integer Programme (Savelsbergh & Sol, 1995)

Decision variables:

- $x_{ij}^k \in \{0, 1\}$: equals 1 if vehicle k travels directly from node i to node j .
- $B_i^k \geq 0$: the time at which vehicle k begins service at node i .
- $L_i^k \geq 0$: the load of vehicle k when it departs node i .

Objective – minimise total travel cost:

$$\min \sum_{k \in K} \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij}^k \quad (\text{PDPTW-OBJ})$$

Constraints:

$$\sum_{k \in K} \sum_{j \in N} x_{ij}^k = 1 \quad \forall i \in P \quad (\text{C1}) \quad (1)$$

$$\sum_{j \in N} x_{0j}^k = 1 \quad \forall k \in K \quad (\text{C2}) \quad (2)$$

$$\sum_{i: 0 \leq i \leq n-1} x_{i, i+1}^k = 1 \quad \forall k \in K \quad (\text{C3}) \quad (3)$$

6.4.2 Exact Methods for PDPTW I

Exact methods for the PDPTW guarantee an optimal solution at the cost of potentially exponential running time. They are practical for instances up to roughly 50–75 requests depending on the tightness of the time windows.

Branch-and-cut (B&C). The Branch-and-Cut method solves a sequence of LP relaxations strengthened by cutting planes, combined with a branch-and-bound tree search. The general scheme is:

- 1 **Solve LP relaxation** at the root node. If the optimal LP solution is integer, stop – it is the optimal PDPTW solution. Otherwise, proceed.
- 2 **Add cuts:** separate violated subtour elimination constraints, capacity cuts, and PDP-specific precedence cuts to tighten the LP relaxation.
- 3 **Branch:** select a fractional variable x_{ij}^k and create two sub-problems (branches): one fixing $x_{ij}^k = 0$ and one fixing $x_{ij}^k = 1$.
- 4 **Prune:** if the LP lower bound of a node exceeds the incumbent upper bound, prune that branch.
- 5 **Repeat** until all nodes are resolved.

6.4.2 Exact Methods for PDPTW II

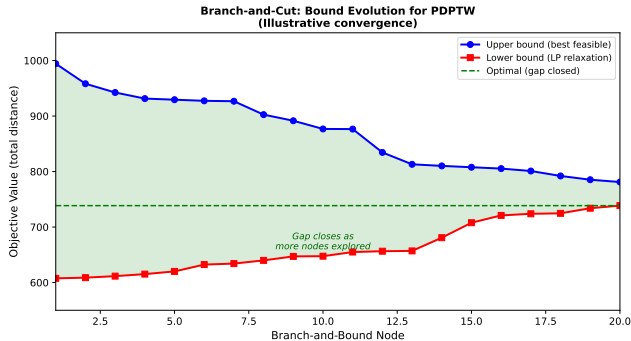


Figure: Illustrative convergence of the branch-and-cut method. The upper bound (best feasible solution found) decreases while the lower bound (LP relaxation) increases as more nodes are explored, until the gap is closed.

6.4.2 Exact Methods for PDPTW III

Key B&C results for PDPTW. Cordeau (2006) proposed a branch-and-cut algorithm for the PDPTW that solved instances with up to 75 requests. The key ingredients were:

- **k -path inequalities:** violated if the current LP solution routes more than k paths through a cut set, beyond what the capacity allows.
- **Clique-based lifted cover inequalities:** tighten capacity constraints by exploiting incompatibility between requests.
- **Branching on 3-cycles:** when many 3-cycle variables are fractional, branching on their indicator variables tends to close the LP gap quickly.

Dynamic programming for single-vehicle PDPTW. When only one vehicle is available, the problem reduces to a sequencing problem. Exact DP formulations track the current time, current load, and the set of requests whose pickup has been made but delivery not yet completed (*open requests*). The state space is exponential in the number of open requests, making DP practical only for small instances.

Column generation / Branch-and-Price. An alternative exact approach formulates the PDPTW as a **set-partitioning problem**:

$$\min \sum_{r \in \Omega} c_r \lambda_r \quad \text{s.t.} \quad \sum_{r \in \Omega} a_{ir} \lambda_r = 1 \quad \forall i, \quad \lambda_r \in \{0, 1\}$$

6.4.2 Exact Methods for PDPTW IV

where Ω is the set of all feasible routes, c_r is the cost of route r , and $a_{ir} = 1$ if route r serves request i . Since $|\Omega|$ is exponentially large, column generation solves the LP relaxation by generating only routes with negative reduced cost – the **pricing sub-problem** – which is itself a shortest-path problem with resource constraints (SPPRC).

6.4.2 Exact Methods for PDPTW V

Complexity summary for exact methods:

Method	Instance size	Key reference
Branch-and-cut	≤ 75 requests	Cordeau (2006)
Branch-and-price	≤ 50 requests	Dumas et al. (1991)
Dynamic programming	≤ 20 requests	Rais et al. (2014)

Key Takeaway: Exact Methods

Branch-and-cut is the most powerful exact method for the PDPTW and can reliably solve instances with up to 75 requests. Beyond this scale, metaheuristics are necessary. The bottleneck for exact methods is the tightness of the LP bound: tight time windows dramatically reduce the search space and allow larger instances to be solved exactly.

6.4.3 Heuristics and Metaheuristics for PDPTW I

Because the PDPTW is NP-hard, practical solvers rely on heuristics and metaheuristics that trade off solution quality for computational efficiency. The goal is to find near-optimal solutions quickly, especially for real-world instances with 100–1000 requests.

Construction heuristics. The most widely used construction approach for the PDPTW is the **parallel or sequential insertion heuristic**:

- 1 Start with m routes, each containing only the depot.
- 2 For each unassigned request, compute the **best feasible insertion** into every existing route – the position that minimises the increase in total cost subject to precedence, capacity, and time-window constraints.
- 3 Insert the request that can be cheaply inserted first (cheapest-first) or most urgently (most constrained first).
- 4 Repeat until all requests are assigned.

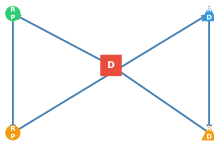
6.4.3 Heuristics and Metaheuristics for PDPTW II

Insertion Heuristic for PDPTW: Step-by-Step

Step 1: Insert Request 1
(cheapest feasible position)



Step 2: Insert Request 2
(best cost increase)



Step 3: Insert Request 3
(final route)

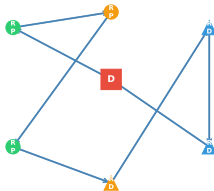


Figure: Three-step insertion heuristic for a PDPTW with 3 requests. Step 1 inserts Request 1 (orange pair) to form the first partial route. Step 2 inserts Request 2 at the best feasible position within the existing route. Step 3 inserts Request 3 (orange) to produce the final solution.

6.4.3 Heuristics and Metaheuristics for PDPTW III

Tabu Search (TS). Tabu Search was applied to the PDPTW by Nanry and Barnes (2000) and later extended by Li and Lim (2003), among others. The core idea is to explore the solution space by systematically making improving or, when no improvement exists, the least-worsening moves, while maintaining a **tabu list** of recently performed moves to avoid cycling.

6.4.3 Heuristics and Metaheuristics for PDPTW IV

Tabu Search Framework for PDPTW

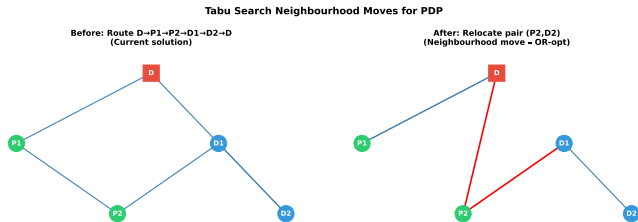
Input: an initial feasible solution s_0 ; tabu tenure T ; maximum iterations $I_{\max}$.

Output: the best solution found s^* .

- ① Set $s \leftarrow s_0$, $s^* \leftarrow s_0$, tabu list $\mathcal{T} \leftarrow \emptyset$.
- ② **For** $i = 1, 2, \dots, I_{\max}$:
 - a. Generate the neighbourhood $\mathcal{N}(s)$ of s by applying all feasible *request-relocation* moves: remove a request (p_i, d_{n+i}) from its current route and reinsert it at the best feasible position in the same or a different route.
 - b. Select the best non-tabu move m^* from $\mathcal{N}(s)$, or accept a tabu move if it satisfies the **aspiration criterion** (the resulting solution is better than s^*).
 - c. Apply m^* : $s \leftarrow s \oplus m^*$.
 - d. Add m^* to $\mathcal{T}$; remove the oldest entry if $|\mathcal{T}| > T$.
 - e. If $f(s) < f(s^*)$ then $s^* \leftarrow s$.
- ③ **Return** s^* .

6.4.3 Heuristics and Metaheuristics for PDPTW V

Neighbourhood structure for PDPTW TS. The most effective neighbourhood for PDPTW TS uses **request relocation**: remove a (pickup, delivery) pair from its current route and reinsert it into another route (or the same route at a different position). The move must maintain feasibility – precedence, capacity, and time windows.



6.4.3 Heuristics and Metaheuristics for PDPTW VI

Figure: Before and after a request-relocation neighbourhood move in PDPTW Tabu Search. The pair (P2, D2) is removed from route 1 and relocated to a different position (shown in red). After the move, both routes must remain feasible.

Why does the tabu list help? In local search without a tabu list, the algorithm tends to cycle between two solutions that are local optima of each other. The tabu list forbids reversing the most recent T moves, forcing the search to explore new regions. Setting T too small allows cycling; T too large makes the search inflexible. Adaptive tabu tenures (randomly varying between $T_{\min}$ and $T_{\max}$) are common in practice.

6.4.3 Heuristics and Metaheuristics for PDPTW VII

Large Neighbourhood Search (LNS) for PDPTW. Ropke and Pisinger (2006) proposed LNS as an effective method for the PDPTW. The algorithm iteratively:

- ① **Destroys** part of the current solution by removing q requests (where q is a *destroy degree*, typically $q \approx 0.1 \cdot n$ to $0.4 \cdot n$).
- ② **Repairs** the partial solution by reinserting the removed requests using a greedy insertion heuristic.
- ③ **Accepts** the new solution according to a simulated annealing criterion: s_{new} is accepted if $f(s_{\text{new}}) < f(s)$, or with probability $\exp(-(f(s_{\text{new}}) - f(s))/T)$ if $f(s_{\text{new}}) \geq f(s)$.

LNS Destroy Operators

- **Random removal:** select q requests uniformly at random.
- **Shaw removal:** remove q requests that are *similar* to each other, where similarity is measured by geographic proximity and time-window overlap. The idea is that similar requests are easier to mutually reassign.
- **Worst removal:** remove the q requests with the highest contribution to the total solution cost (the most expensive requests).

6.4.3 Heuristics and Metaheuristics for PDPTW VIII

Adaptive LNS (ALNS). Ropke and Pisinger (2006) extended LNS to the **Adaptive Large Neighbourhood Search (ALNS)** by maintaining a *pool* of destroy and repair operators, and dynamically adjusting the probability of selecting each operator based on its historical performance:

- Each operator o is assigned a score π_o that accumulates reward whenever the operator contributes to a new best solution, an improving solution, or an accepted non-improving solution.
- After every σ iterations (a *segment*), the selection probability of each operator is updated:

$$p_o \leftarrow \frac{w_o}{\sum_{o'} w_{o'}}, \quad w_o \leftarrow (1 - r)w_o + r \frac{\pi_o}{\theta_o}$$

where w_o is the weight, $r \in (0, 1)$ is a reaction factor controlling how quickly weights adapt, and θ_o is the number of times operator o was used during the segment.

- Operators that consistently find better solutions receive higher probability of being selected in subsequent iterations.

Why ALNS works. Different destroy operators are effective for different types of problem structure. Random removal provides diversification; Shaw removal provides intensification around related requests; worst removal targets high-cost areas. Adapting probabilities ensures the algorithm exploits whichever strategy is most effective for the current instance.

6.4.3 Heuristics and Metaheuristics for PDPTW IX

ALNS Numerical Illustration: Score Update

Suppose the algorithm has 3 destroy operators: Random (R), Shaw (S), Worst (W). After one segment of $\sigma = 50$ iterations, the collected statistics are:

Operator	θ_o (uses)	π_o (score)	w_o (old)	w_o (new, $r = 0.2$)
Random (R)	18	24	1.00	$0.8 \times 1.00 + 0.2 \times (24/18) = 1.067$
Shaw (S)	17	38	1.00	$0.8 \times 1.00 + 0.2 \times (38/17) = 1.247$
Worst (W)	15	12	1.00	$0.8 \times 1.00 + 0.2 \times (12/15) = 0.960$

The selection probabilities become: $p_R = 1.067/3.274 = 0.326$; $p_S = 1.247/3.274 = 0.381$; $p_W = 0.960/3.274 = 0.293$. Shaw removal, having found the most improvements, now has the highest selection probability.

6.4.3 Heuristics and Metaheuristics for PDPTW X

Branch-and-Cut integrated with heuristics. Ropke et al. (2007) combined branch-and-cut with LNS in a so-called **matheuristic** framework: the LNS is used to find a good upper bound, which is then passed to the B&C as the incumbent, greatly reducing the B&C search tree.

POPMUSIC. For very large PDPTW instances (several hundred requests), the POPMUSIC framework (Partial Optimisation Metaheuristic Under Special Intensification Conditions) by Taillard et al. has been adapted: the problem is decomposed into overlapping sub-problems of manageable size, each sub-problem is optimised by a fast metaheuristic, and the solutions are stitched back together.

6.4.3 Heuristics and Metaheuristics for PDPTW XI

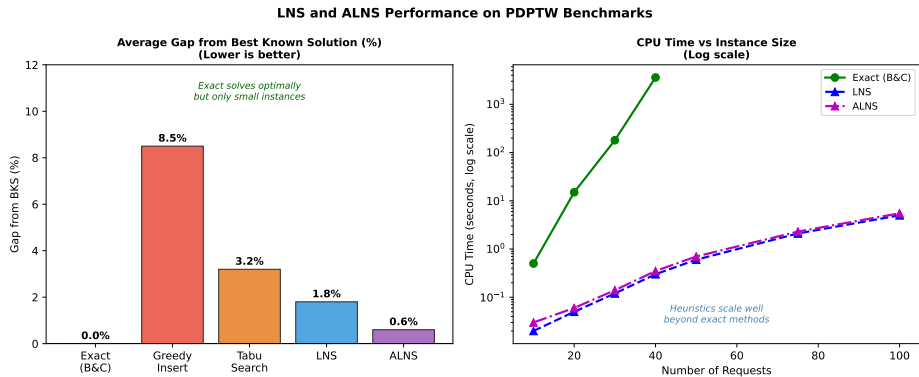


Figure: Left: average gap from best known solution for different methods on PDPTW benchmarks. ALNS achieves solutions within 0.6% of BKS. Right: CPU time vs instance size (log scale); exact B&C is practical only for small instances while LNS and ALNS scale to hundreds of requests.

6.4.3 Heuristics and Metaheuristics for PDPTW XII

Key Takeaway: Metaheuristics for PDPTW

ALNS is currently the leading metaheuristic for the PDPTW, offering the best balance of solution quality and scalability. The key design choice is the set of destroy/repair operators: combining random, similarity-based (Shaw), and worst-case removal with adaptive operator selection has proven highly effective across diverse benchmark sets.

6.4.4 Single-Vehicle PDPTW I

When the fleet consists of a single vehicle, the PDPTW reduces to the **Pickup-and-Delivery TSP with Time Windows (PDTSPW)**. This problem has been studied both as a theoretical object and as a sub-problem in column generation approaches to the multi-vehicle PDPTW.

Single-vehicle exact algorithm. Ruland and Rodin (1997) proposed a branch-and-cut algorithm specifically for the single-vehicle case. The key observation is that for a single vehicle, the constraint that the same vehicle serves both endpoints of each request is automatically satisfied. The remaining difficulty is the combination of:

- Precedence constraints: pickup i before delivery $n + i$.
- Capacity constraints: load $\in [0, Q]$ throughout.
- Time window constraints: service within $[e_i, l_i]$ at each node.

6.4.4 Single-Vehicle PDPTW II

Algorithms for the single-vehicle PDTSP (no time windows). Without time windows, the problem simplifies to the PDTSP. Hernandez-Perez and Salazar-Gonzalez proposed a branch-and-cut that uses a clever reformulation: they observe that the PDTSP is equivalent to the TSP on a reduced graph where infeasible arcs (those that would violate precedence or capacity) are removed. Their algorithm solves instances with up to 40 nodes.

Key Takeaway: Single-Vehicle PDPTW

The single-vehicle PDPTW is important as a sub-problem in column generation for the multi-vehicle case. Efficient algorithms for it (particularly those that generate optimal routes quickly) are critical to the scalability of branch-and-price methods for the full multi-vehicle PDPTW.

6.4.5 Multiple-Vehicle PDPTW I

In the multi-vehicle setting, a fleet K of identical vehicles must collectively serve all n requests. This is the standard industrial setting and the most studied variant in the literature.

6.4.5 Multiple-Vehicle PDPTW II

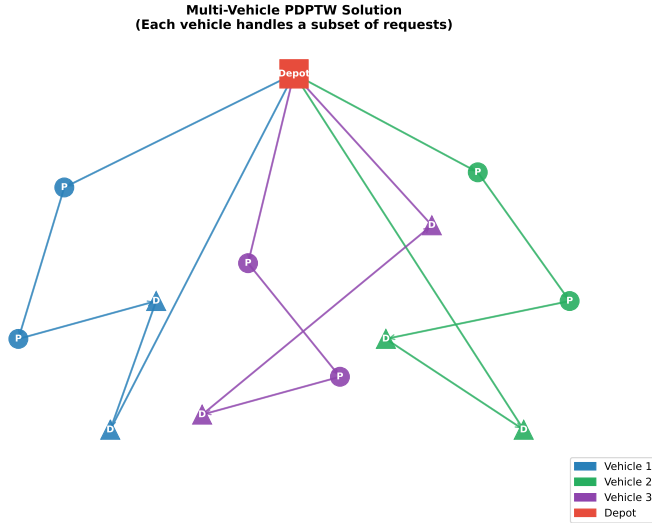


Figure: A multi-vehicle PDPTW solution with 3 vehicles (blue, green, purple). Each vehicle serves a subset of $_{40/59}$

Python: PDPTW Feasibility Check and Greedy Construction

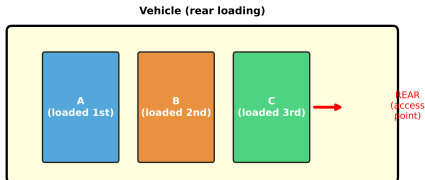
```
1  """
2  PDPTW feasibility checker and greedy insertion heuristic.
3  Assumes symmetric travel cost/time: dist[i][j] == dist[j][i].
4  """
5  from dataclasses import dataclass, field
6  from typing import List, Tuple, Optional
7  import math
8
9  @dataclass
10 class Request:
11     idx: int          # request id (1..n)
12     pickup: int       # pickup node index
13     delivery: int     # delivery node index
14     load: float       # goods quantity
15     e_p: float        # earliest service at pickup
16     l_p: float        # latest service at pickup
17     e_d: float        # earliest service at delivery
18     l_d: float        # latest service at delivery
19
20 def is_feasible(route: List[int], requests: dict, dist: List[List[float]],
21               capacity: float, service_time: float = 0.0) -> bool:
22     """
23     Check if a route is feasible for PDPTW.
24     route: list of node indices including depot (0) at start and end.
25     requests: dict mapping pickup_node -> Request object.
26     Returns True if precedence, capacity, and time windows are all satisfied.
27     """
```

6.5 Problems with Loading Constraints I

In practice, goods transported by vehicles are physical three-dimensional objects. The order in which goods are loaded onto a vehicle determines whether they can be unloaded at each delivery stop without disturbing other cargo. This gives rise to **loading-constrained PDPs** – problems in which both the routing and the loading plan must be co-optimised.

Loading Constraints: LIFO vs FIFO

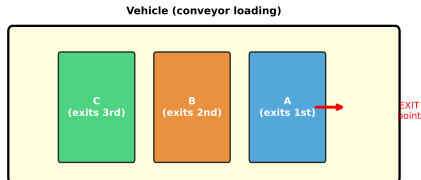
LIFO (Last-In First-Out)
- rear loading, stack policy



Must unload C, then B, then A (stack order)

LIFO: delivery sequence = reverse of loading sequence

FIFO (First-In First-Out)
- side/conveyor loading



A exits first (loaded first), then B, then C

FIFO: delivery sequence = same as loading sequence

6.5 Problems with Loading Constraints II

Figure: Left: LIFO loading – items loaded first (deepest inside the vehicle) must be delivered last; items near the rear are delivered first. Right: FIFO loading – items loaded first exit first (conveyor-style). The delivery sequence on the route must respect the physical loading order.

6.5 Problems with Loading Constraints III

Three types of loading constraints:

- ① **LIFO (Last-In First-Out):** the vehicle is accessed from the rear only (like a stack). The most recently loaded item must be unloaded first. This means the *delivery sequence must be the reverse of the loading sequence*: if item *A* is loaded before item *B*, then *B* must be delivered before *A*.
- ② **FIFO (First-In First-Out):** the vehicle is a conveyor – items loaded first exit first. The delivery sequence must *match* the loading sequence.
- ③ **3D Loading:** items have explicit three-dimensional extents (length, width, height) and must physically fit inside the container at every stage of the route. This requires solving a bin-packing sub-problem at each delivery stop.

6.5 Problems with Loading Constraints IV

LIFO Constraint: Formal Statement

Let σ be the route sequence of customers visited by a vehicle, and let π be the loading order (order in which items are placed onto the vehicle). The LIFO constraint states:

For any two items i and j carried by the same vehicle, if i is delivered before j (i.e., i comes before j in the route), then j must have been loaded before i (i.e., j precedes i in the loading order).

Equivalently: $\text{position}(\text{deliver } i) < \text{position}(\text{deliver } j) \Rightarrow \text{position}(\text{load } j) < \text{position}(\text{load } i)$.

6.5 Problems with Loading Constraints V

PDPTW with LIFO constraints. Carrabs, Cordeau, and Laporte (2007) studied the **PDPTW-L** (PDPTW with LIFO constraints) and showed that:

- LIFO constraints can be enforced without explicitly tracking the loading order: it suffices to check that the route does not contain any *interleaving* pair. Two requests i and j are **interleaved** if the route visits them in the order p_i, p_j, d_i, d_j (or p_j, p_i, d_j, d_i). LIFO feasibility requires that no such interleaving occurs.
- Equivalently: for every pair of requests on the same vehicle, the pickup-delivery intervals must be either *nested* (one contained within the other) or *disjoint* (non-overlapping).

6.5 Problems with Loading Constraints VI

LIFO Interleaving: Worked Example

Suppose a route visits nodes in order: Depot, P_1 , P_2 , D_1 , D_2 , Depot.

- Requests: (P_1, D_1) and (P_2, D_2) .
- Position of $P_1 = 1$, $P_2 = 2$, $D_1 = 3$, $D_2 = 4$.
- The sequence $P_1 < P_2 < D_1 < D_2$ means the intervals $[1, 3]$ and $[2, 4]$ *overlap but neither is contained in the other* – this is **interleaved**.
- A LIFO-feasible alternative: Depot, P_1 , P_2 , D_2 , D_1 , Depot gives intervals $[1, 4]$ and $[2, 3]$ – interval $[2, 3]$ is **nested** inside $[1, 4]$. This satisfies LIFO: item 2 (loaded after item 1 at P_2) is delivered first at D_2 , then item 1 at D_1 . ✓

6.5 Problems with Loading Constraints VII

Algorithms for PDPTW-L. Carrabs et al. (2007) designed a Branch-and-Cut algorithm for PDPTW-L that incorporates specific valid inequalities for LIFO feasibility. The branch-and-cut was able to solve instances with up to 25 requests.

For larger instances, heuristic approaches are necessary:

- **Tabu Search:** the neighbourhood moves must be restricted to those that maintain LIFO feasibility. A swap move that creates an interleaving is rejected.
- **LNS with LIFO repair:** after removing a subset of requests, the repair heuristic inserts them at positions that maintain the nesting property.
- **Dynamic programming:** for a single vehicle, DP can track the set of currently open (picked up but not yet delivered) requests and ensure LIFO ordering at each step.

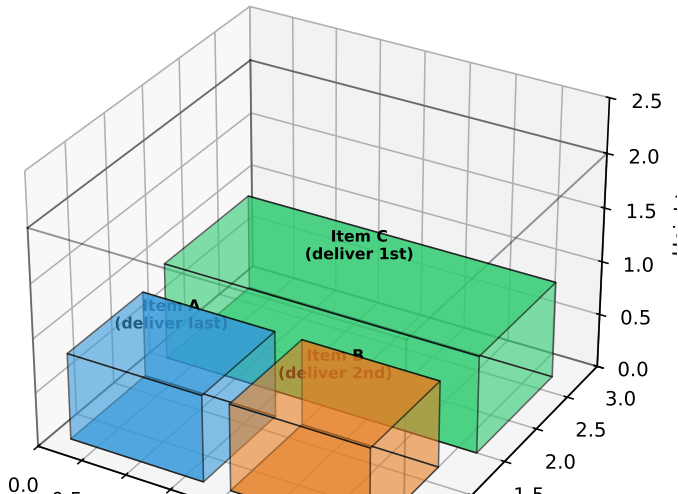
PDPTW with FIFO constraints. The PDPTW-F (FIFO variant) has analogous structure: instead of nesting, the pickup-delivery intervals must be *compatible* in the FIFO sense. Ladier and Alpan (2016) and others have shown that FIFO constraints arise naturally in conveyor-loading contexts such as airport baggage handling.

6.5 Problems with Loading Constraints VIII

3D Loading PDP. The most complex loading variant requires that items be physically packed into the vehicle's container in three dimensions. At every delivery stop, the item to be unloaded must be accessible from the door without requiring other items to be moved.

6.5 Problems with Loading Constraints IX

3D Loading Constraints in PDP
(Items must fit and be accessible at each delivery stop)



6.5.1 One-to-One Problems with Loading Constraints I

The one-to-one structure is the most natural setting for loading-constrained PDPs, since each request has a dedicated item to be picked up and delivered. Research has focused on two specific settings:

Single-vehicle PDPTW with LIFO. Carrabs, Cordeau, and Laporte (2007) proposed a Branch-and-Cut based on the following key result: LIFO feasibility of a route can be verified in $O(n)$ time by scanning the route and maintaining a stack. If, when attempting to deliver item i , the top of the stack is not i , the route is LIFO-infeasible.

Algorithm (LIFO Feasibility Check):

- ① Initialise an empty stack S .
- ② Scan the route from the depot to the end:
 - If the current node is a **pickup** p_i : push i onto S .
 - If the current node is a **delivery** d_i : if $\text{top}(S) = i$, pop S ; else the route is **LIFO-infeasible**.
- ③ If the stack is empty at the end, the route is LIFO-feasible.

6.5.1 One-to-One Problems with Loading Constraints II

LIFO Stack Check: Example

Route: Depot, P_1 , P_3 , D_3 , P_2 , D_2 , D_1 , Depot.

Node	Action	Stack
P_1	Push 1	$\{1\}$
P_3	Push 3	$\{1, 3\}$
D_3	Top = 3, Pop	$\{1\}$
P_2	Push 2	$\{1, 2\}$
D_2	Top = 2, Pop	$\{1\}$
D_1	Top = 1, Pop	$\{\}$

Stack empty at end: route is **LIFO-feasible**. ✓ The structure shows nested intervals: $[P_1, D_1]$ contains $[P_2, D_2]$, which is disjoint from $[P_3, D_3]$.

6.5.1 One-to-One Problems with Loading Constraints III

One-to-Many-to-One with Loading Constraints. In the 1-M-1 setting, Salazar-Gonzalez (2014) considered a variant where the same vehicle delivers goods from the depot (forward logistics) and collects returned goods (reverse logistics) on the same tour. LIFO constraints arise because the vehicle is loaded at the depot in a specific order that constrains the delivery sequence.

Incorporating loading into the MIP. In principle, loading constraints can be incorporated directly into the integer programme by adding binary variables $l_{ij} \in \{0, 1\}$ indicating whether item i is loaded before item j , with constraints linking l_{ij} to the route variables x . However, this leads to a significantly larger MIP. The preference in practice is to use the stack-based feasibility check as an efficient oracle within a metaheuristic.

Key Insight: LIFO as a Stack

The key algorithmic insight for LIFO-constrained PDPs is that LIFO feasibility is equivalent to the route forming a valid parenthesisation when pickup nodes are treated as opening parentheses and delivery nodes as closing parentheses. This allows $O(n)$ feasibility checking and simple neighbourhood filtering in TS and LNS.

6.6 Conclusions and Future Research Directions I

This chapter has presented the main variants of Pickup-and-Delivery Problems for goods transportation, classified by the relationship between origins and destinations (M-M, 1-M-1, 1-1) and by additional constraints (time windows, loading).

Summary of major algorithms:

Problem	Best Exact	Best Heuristic	Max Size
1-PDTSP (M-M, 1 vehicle)	B&C (H-P & S-G)	VNS, GRASP	200 nodes
M-PDTSP (M-M, multi-veh)	B&C	GRASP+VND (Qu&Bard)	300 nodes
VRPSPD (1-M-1)	B&C	Tabu Search, LNS	200 cust.
PDPTW (1-1, multi-veh)	B&C (Cordeau)	ALNS (Ropke&Pisinger)	1000 req.
PDPTW-L (1-1 + LIFO)	B&C	TS, LNS	100 req.
3L-PDPTW (3D loading)	—	2-level meta.	50 req.

Dominant research themes:

6.6 Conclusions and Future Research Directions II

- ① **Exact methods** have matured significantly, with branch-and-cut capable of solving mid-size PDPTW instances (up to 75 requests) in reasonable time. Column generation and branch-and-price extend this somewhat further.
- ② **ALNS and its variants** have become the standard metaheuristic for PDPs, largely displacing Tabu Search for single-objective problems. The key insight is the adaptive operator selection mechanism.
- ③ **Loading constraints** (LIFO, FIFO, 3D) have received increasing attention as the practical importance of physical loading feasibility is recognised. The coupling of routing and packing sub-problems remains a significant challenge.

6.6 Conclusions and Future Research Directions III

Open research directions:

- 1 **Dynamic PDPs:** in many real applications (courier delivery, ride-hailing), requests arrive in real time. Reoptimisation strategies that update the routing plan as new requests arrive, with minimal disruption to ongoing routes, remain an active research frontier.
- 2 **Stochastic PDPs:** demand quantities or travel times may be uncertain. Robust and stochastic optimisation formulations for PDPs are less mature than for standard VRPs.
- 3 **Multi-commodity PDPs:** when different items must not be mixed (e.g., food-grade and non-food-grade goods), the loading constraints become a multi-commodity assignment problem.
- 4 **Green PDPs:** minimising fuel consumption or CO₂ emissions rather than distance leads to different optimal routes, especially when vehicle weight (and thus fuel consumption) changes as goods are picked up and delivered.
- 5 **Crowd-sourced delivery:** platforms that use ordinary citizens as carriers (as in Uber Freight) create large-scale PDPs with hundreds of thousands of requests and require fundamentally new algorithmic approaches.

6.6 Conclusions and Future Research Directions IV

- 6 **Integration with inventory management:** joint optimisation of routing and inventory replenishment decisions leads to PDPs with vendor-managed inventory (VMI) structure, where the carrier also decides delivery quantities.

Chapter Summary

Pickup-and-Delivery Problems for goods transportation are a rich and practically important family of combinatorial optimisation problems. The PDPTW has emerged as the canonical problem, with ALNS as the dominant metaheuristic and branch-and-cut as the leading exact method. Loading constraints (LIFO, FIFO, 3D) add physical realism at the cost of significantly increased algorithmic complexity. Future research will increasingly focus on dynamic, stochastic, and large-scale variants driven by the growth of e-commerce and sharing-economy platforms.

Selected Bibliography I

- **Battarra, Cordéau, Iori (2014)**. Chapter 6: Pickup-and-Delivery Problems for Goods Transportation. In: *Vehicle Routing: Problems, Methods, and Applications* (2nd ed.), SIAM.
- **Savelsbergh & Sol (1995)**. The general pickup and delivery problem. *Transportation Science*, 29(1), 17–29.
- **Cordeau (2006)**. A branch-and-cut algorithm for the dial-a-ride problem. *Operations Research*, 54(3), 573–586.
- **Ropke & Pisinger (2006)**. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.
- **Li & Lim (2003)**. A metaheuristic for the pickup and delivery problem with time windows. *International Journal of Artificial Intelligence Tools*, 12(2), 173–186.
- **Hernandez-Perez & Salazar-Gonzalez (2004)**. A branch-and-cut algorithm for a traveling salesman problem with pickup and delivery. *Discrete Applied Mathematics*, 145(1), 126–139.
- **Carrabs, Cordeau & Laporte (2007)**. Variable-neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading. *INFORMS Journal on Computing*, 19(4), 618–632.
- **Montane & Galvao (2006)**. A tabu search algorithm for the vehicle routing problem with simultaneous pick-up and delivery service. *Computers & Operations Research*, 33(3), 595–619.
- **Dumas, Desrosiers & Soumis (1991)**. The pickup and delivery problem with time windows. *European Journal of Operational Research*, 54(1), 7–22.

Selected Bibliography II

- **Ruland & Rodin (1997)**. The pickup and delivery problem: Faces and branch-and-cut algorithm. *Computers & Mathematics with Applications*, 33(12), 1–13.
- **Qu & Bard (2012)**. A GRASP with adaptive large neighborhood search for pickup and delivery problems with transshipment. *Computers & Operations Research*, 39(10), 2439–2456.
- **Bianchessi & Righini (2007)**. Heuristic algorithms for the vehicle routing problem with simultaneous pick-up and delivery. *Computers & Operations Research*, 34(2), 578–594.
- **Nanry & Barnes (2000)**. Solving the pickup and delivery problem with time windows using reactive tabu search. *Transportation Research Part B*, 34(2), 107–121.